



Efficient algorithms for deriving complete frequent itemsets from frequent closed itemsets

Cheng-Wei Wu¹ · JianTao Huang¹ · Yun-Wei Lin¹ · Chien-Yu Chuang¹ · Yu-Chee Tseng²

Accepted: 22 December 2020 / Published online: 11 April 2021

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract

When *mining frequent itemsets* (abbr. *FIs*) from dense datasets, it usually produces too many itemsets and results in the mining task to suffer from a very long execution time and high memory consumption. *Frequent closed itemset* (abbr. *FCI*) is a compact and lossless representation of FI. Mining FCIs can not only reduce the execution time and memory usage, but also reserve the complete information of FIs derived from FCIs. Although many studies have been proposed with various efficient methods for mining FCIs, few of them have developed algorithms for efficiently deriving FIs from FCIs. In this work, we propose two efficient algorithms named *DFI-List* and *DFI-Growth* for efficiently deriving FIs from FCIs. The both algorithms adopt depth-first search and divide-and-conquer methodology to derive all the FIs. *DFI-List* efficiently derives all the FIs with a vertical index structure called *Cid List*. *DFI-Growth* compresses the information of FCIs into tree structures and applies pattern-growth strategy to derive FIs from the trees. Empirical experiments show that *DFI-List* is the most efficient and scalable algorithm on the dense datasets. For example, when the minimum support threshold is set to 50% on the Chess dataset, *DFI-List* runs faster than *LevelWise* (Pasquier et al. Inf Syst 24(1): 25–46, 1999b) over 100 times. As for *DFI-Growth*, it is the most stable and memory efficient algorithm on the sparse datasets. Both *DFI-Growth* and *DFI-List* are superior to the state-of-the-art algorithm (Pasquier et al. Inf Syst 24(1): 25–46, 1999b) in terms of execution time.

Keywords Frequent itemset mining · Frequent closed itemset mining · Lossless and condensed representation · Deriving algorithms

1 Introduction

Association rule mining [1, 2, 5, 9, 10, 29] is one of the important technology in big data mining. It can be used to discover associated items that frequently co-occur in databases and express their relationships in the form of rules. Association rule mining can be generally divided into two stages, which are *frequent itemset mining* and *association rule generation*. The stage of frequent itemset mining (abbr. *FIM*) aims at finding all the sets of items (i.e., itemsets) that co-occur frequently in databases. The found itemsets are referred to the *frequent itemsets* (abbr.

FIs). The stage of *association rule generation* focuses on deriving rules as well as their supports and confidences from the found FIs. If the support and confidence of a rule is no less than a *minimum support threshold* and a *minimum confidence threshold* defined by the user, the rule is called *association rule*. Association rule mining has been widely extended to various applications and mining frameworks, such as *utility mining* [12, 13, 24, 27, 28], *uncertain data mining* [15], *text mining* [17], *web mining* [31], *biomedicine* [25] and even *smart manufacturing* [27].

It is widely recognized that the main computational cost of association rule mining lies in the stage of frequent itemset mining. So far, many algorithms have been proposed for improving the efficiency of mining FIs, such as *Apriori* [1], *FP-Growth* [10], *Eclat* [29], *negFIN* [2] and *dFIN* [5]. As indicated in the experiments of previous studies, when mining FIs from sparse data, these algorithms may have fairly performance. However, while mining FIs from dense data, they may encounter the bottleneck of long execution time and even unable to complete due to memory exhaustion. This is because the correlations between items

This article belongs to the Topical Collection: *Special issue on Artificial intelligence in practice - from theory to application*
Guest Editors: Franz Wotawa, Gerhard Friedrich and Ingo Pill

✉ Cheng-Wei Wu
wucw@niu.edu.tw

Extended author information available on the last page of the article.

in sparse data are relatively weak, and items appear less frequently together in records. In addition, in sparse data, the lengths of transactions are relatively short and there are fewer FIs. In this circumstance, the *downward closure property* [1, 10] may bring a good utility. The downward closure property states that if an itemset is infrequent, all its supersets are infrequent. In sparse dataset, different items co-occur less frequently in records. Therefore, when mining FIs in sparse data, the conventional algorithms can rely on this property to effectively narrow down the search space and work well. However, in dense data, the correlations between items are relatively strong, and items co-occur more frequently in records. Besides, transactions in dense data are relatively long and there are a large amount of FIs. In this situation, the downward closure property may be unable to work well for largely reducing the search space. This is because the more patterns that are needed to be discovered, the more computational costs are required by the mining algorithms. As a consequent, when mining FIs from dense data, the conventional algorithms always suffer from slow speed and a huge amount of memory consumption.

Instead of directly mining all FIs, several studies have focused on *mining concise representation of FIs*, including *generator* [21], *free itemset* [3], *non-derivable itemset* [4], *maximal itemset* [8, 16]. In general, the concise representation of FIs is a small subset of all the FIs, where the number of patterns is often much smaller than that of all FIs. Since the number of patterns to be discovered is largely reduced, the computational cost for the mining task is also significantly reduced. Although the above studies [3, 4, 8, 16, 21] find out some interesting patterns from the databases, they only consider mining of concise representation rather than the lossless representation. In other words, the patterns mined by the above studies do not retain the complete information of FIs, including all the FIs and their supports. *Frequent closed itemset* (abbr. *FCI*) is a *lossless and concise representation* [14, 18, 20, 23, 26, 30] of FIs. Mining only the FCIs will not lose any information about FIs. Through the correct and efficient deriving algorithm, all the FIs and their supports can be derived from FCIs. Although FCI mining is a popular research direction, seldom of existing studies have developed efficient algorithms for deriving FIs from FCIs. In this paper, we name such algorithm as *deriving algorithm*.

As far as we know, so far there is only one deriving algorithm, which was proposed by Pasquier et al. [21] in the year 1999. In this paper, we call it as the *LevelWise* algorithm. The LevelWise algorithm adopts a level-wise and top-down search scheme for deriving FIs. It derives the longest FIs to the shortest FIs level by level. During the deriving process, LevelWise needs to keep all the FIs of

length k and $(k-1)$. Therefore, when there are many itemsets need to be kept in memory, LevelWise may spend a lot of time on searching and matching patterns, leading it to suffer from slow execution speed. Moreover, a large number of itemsets kept in two levels will also make LevelWise to consume a lot of memory resource. In even worse case, LevelWise may fail to complete the deriving task due to running out of memory.

However, it is important to develop correct and efficient deriving algorithms. Below we give three scenarios where the deriving algorithms can be applied. Firstly, When the traditional FIM algorithms cannot directly mine all FIs from high-dimensional datasets, users can consider to mine FCIs first, and then apply the deriving algorithms to recover FIs from FCIs. In biomedical applications, such as microarray and gene expression analytics, the dataset is usually quite large and consists of very high dimension. In this situation, if the algorithms fail to find FIs but success to find FCIs, users can alternatively try to use the deriving algorithms to recover all FIs from FCIs. Secondly, when the users only retain the previously discovered FCIs, but does not retain the original databases, the users cannot directly use the FIM algorithms to derive FIs from FCIs. Because the traditional FIM algorithms have not designed for the deriving task. In this case, the users can use the deriving algorithms to obtain all the FIs. Obtaining the complete information of FCIs and FIs allows users to conduct more applications and analytics. Thirdly, when the original databases and the previously mined FCIs are retained, but FIs are not been mined before and the databases are very large. If the users want to obtain FIs, they can use the FIM algorithms. However, the algorithms must re-scan the large original databases and calculate support for each FI, which may result in a considerable time and calculation cost. In this case, if the number of FCIs is not large, all the FIs can be fast generated by using the deriving algorithms without any scan of the original databases, which saves a lot of time and calculation cost.

In the practical application of data mining, users may encounter the above-mentioned difficulties. In view of this, this paper proposes a series of efficient deriving algorithms to provide users with more diversified solutions. The main contributions of this research are summarized as follows:

- Since the year 1999, there are no studies dedicated to developing advancing deriving algorithms. This paper takes the lead in re-exploring this research direction and proposes two novel frameworks for the deriving task, named *DFI-List (Deriving Frequent Itemsets from FCIs based on Cid List)* and *DFI-Growth (Deriving Frequent Itemsets from FCIs based on Pattern Growth)*. The main features of the former are simple, easy to be implemented as a disk-based algorithm, and generally

occupies less memory usage for dense data. As for the latter one, it takes good use of in-memory tree structures and processing technologies. Its execution speed and memory usage are generally better than the current best method LevelWise [21].

- We propose the DFI-List algorithm. Its main design concept is to use a vertical index structure called *Cid List* to store FCIs. The *Cid List* structure records the itemsets and their *CidSeqs*. The *CidSeq* of an itemset X is a sequence of the IDs of the FCIs containing X . Given the *CidSeqs* of any two arbitrary itemsets X and Y , through a simple intersection operation on the their *CidSeqs*, DFI-List can efficiently calculate the *CidSeq* and the support of the itemset $Z = X \cup Y$. With the divide-and-conquer and recursion strategies, DFI-List allows to enumerating the complete FIs along with their *CidSeqs* in an efficient way. Moreover, many operations in DFI-List can be performed with secondary memory (e.g., disk and files). When the main memory space is limited or internal tree structures cannot be fully feed into the memory, DFI-List can be considered and it is a good alternative solution for the deriving task.
- In our preliminary study [11], we proposed the DFI-Growth algorithm. In this paper, we conduct a more complete description and richer experimental evaluation for the algorithm. The main design concept of DFI-Growth is to compress FCIs into FP-Tree-like structures, and use pattern-growth and recursion techniques to generate FIs from the trees. Since traditional FP-Tree cannot be directly applied to deriving FIs from FCIs, we modify it and make it applicable to the deriving task. In addition, we propose the *maximum support replacement* (abbr. *MSR*) strategy, which is applied in the construction phase of the modified FP-Trees, so that the modified FP-Trees can maintain the correct information of FIs and FCIs. Furthermore, we propose the *maximum support selection* (abbr. *MSS*) strategy, which is applied in the pattern-growth phase for efficiently deriving complete FIs.
- We design a series of experiments and compare the proposed algorithms with the current best algorithm LevelWise [21]. We use not only simulated data but also real-life data for the performance evaluation. In the experimental results, among all the compared algorithms, DFI-List uses the least amount of memory and execution time on dense datasets. For instance, on Mushrooms dataset, LevelWise fails to complete the task under low minimum support thresholds, but DFI-List takes only 35 seconds and still has good memory scalability. Furthermore, DFI-Growth outperforms LevelWise [21] in memory space and execution speed. More impressively, the execution speed of DFI-Growth can be faster than the current

best algorithm over 1,000 times in some cases. Taking an example, on the real dataset Chess, at the 40% minimum support threshold, LevelWise takes more than 43,200 seconds, while DFI-Growth finishes the task in only 20 seconds.

2 Basic concept and definitions

In the followings, we define the problem statement of the deriving task and introduce the basic concept and definitions about frequent (closed) itemsets. For more details about frequent (closed) itemsets, readers can refer to the studies [2, 5, 7, 13, 23]. Let $V = \{I_1, I_2, \dots, I_{|V|}\}$ be a finite set of distinct *items*. A *database* $D = \{R_1, R_2, \dots, R_{|D|}\}$ contains several transactions. Every transaction R_d ($1 \leq d \leq |D|$) in the database is a subset of V and has a unique transaction identification d . An *itemset* is a set of items. If all the items inside an itemset X are contained in a transaction R_d , X is said to *be contained in* R_d and it is denoted as $X \subseteq R_d$.

Definition 1 (Length of an itemset) Let $X = \{I_1, I_2, \dots, I_K\}$ be an itemset consisting of K distinct items, then the *length of* X is defined as $|X| = K$.

Definition 2 (Tidset of an itemset) The *Tidset* of an itemset X is defined as the set of identifications of all the transactions containing X in D . We use the notation $R(X)$ to represent the *Tidset* of X .

Definition 3 (Support count of an itemset) The *support count of an itemset* X is defined as the total number of transactions containing X and denoted as $SC(X)$. Equivalently, $SC(X) = |R(X)|$.

Definition 4 (Support of an itemset) The *support of an itemset* X is denoted as $SP(X)$ and is defined as the ratio of the support count of X to the total number of transactions in D . Equivalently, $SP(X) = SC(X)/|D|$.

Definition 5 (Frequent itemset) An itemset X is called *frequent itemset* if and only if $SP(X)$ is no less than a user-defined *minimum support threshold* $\theta_r\%$ ($0 < \theta_r \leq 100$). Otherwise, X is called *infrequent itemset*. An equivalent definition is that X is frequent if and only if $SC(X) \geq \theta_e$, where θ_e is called *minimum support count threshold* and defined as $|D| \times \theta_r\%$.

Definition 6 (Superset and subset) . Let X and Y be two itemsets. If all the items in X are included in Y , X is called *subset of* Y and Y is called *superset of* X . If $X \neq Y$ and X is a subset of Y , then X is called *proper subset of* Y and Y is called *proper superset of* X .

Property 1 (Downward closure property [1, 10]) For any frequent itemset X , any subset of X is frequent. For any infrequent itemset Y , any supersets of Y is infrequent.

Definition 7 (Closure of an itemset) The *closure* of an itemset X is denoted as $Clo(X)$ and defined as $\bigcap_{d \in R(X)} R_d$, where $Clo(X)$ returns the largest superset of Y in D such that $X \subseteq Y$ and $SC(X) = SC(Y)$. For example, the closure of the itemset $\{T\}$ is $Clo(\{T\}) = R_1 \cap R_3 \cap R_5 \cap R_6 = \{ACTW\} \cap \{ACTW\} \cap \{ACDTW\} \cap \{CDT\} = \{CT\}$.

Property 2 For any itemset X , the support count of X is equal to the support count of its closure. Equivalently, $SC(X) = SC(Clo(X))$ and $|R(X)| = |R(Clo(X))|$.

Definition 8 (Closed itemset) An itemset X is called *closed* itemset if and only if there exists no proper superset Y such that $X \subset Y$ and $SP(X) = SP(Y)$. Otherwise, X is called *non-closed* itemset. Moreover, if $Clo(X) = X$, then X is a closed itemset. Otherwise, X is non-closed.

For instance, considering the database of Table 1, the itemset $\{C\}$ is a closed itemset because $Clo(\{C\}) = \{C\}$. On the contrary, the itemset $\{T\}$ is non-closed because the closure of $\{T\}$ is $Clo(\{T\}) = \{CT\}$ and $Clo(\{T\}) \neq \{T\}$.

Definition 9 (Frequent closed itemset) . A *closed* itemset X is called *frequent closed* itemset if and only if the $SP(X) \geq \theta_r\%$. Otherwise, if $SP(X) < \theta_r\%$, X is an infrequent closed itemset.

Problem statement Let C be a complete set of frequent closed itemsets with their supports in a database D . Given the set C , the problem to be solved for the deriving task is to correctly and efficiently derive the complete set of frequent itemsets with their supports from C .

3 Related work

In this section, we review the studies related to this work, which are divided into three aspects: (1) frequent itemset

mining, (2) frequent closed itemset mining, and (3) the deriving algorithms.

3.1 Frequent itemset mining

Frequent itemset (abbr. *FI*) mining has been extensively studied. *Apriori* [1] is a well-known algorithm for mining FIs. The Apriori algorithm is a breadth-first search algorithm. Its main idea is to generate candidate itemsets of length k ($k \geq 1$) and then scans the database to calculate their supports to identify FIs. During the mining process, invalid candidates are pruned by applying the *downward closure property* [1, 10]. Suppose that the length of the longest FI is M , Apriori has to scan the databases at least M times for finding all the FIs. If the database contains many transactions, it may spend a lot of time on repeatedly scanning the database. Moreover, when Apriori is going to count the support of a candidate itemset, it may take time to search for the position of the candidate itemset, and a large amount of generated candidates also often leads to memory shortage problems. Another cleverly designed algorithm is *FP-Growth* [10]. Its most well-known feature is that it does not need to generate any candidate itemsets during the mining process and it only needs to scan the original database twice. The main idea of the algorithm is to compress the database into a special structure called *FP-Tree*, and then performs a *pattern-growth* strategy to efficiently generate FIs from the trees. In general, the performance of the pattern-growth algorithms is better than the Apriori-like algorithms. Both Apriori and FP-Growth are often used as benchmarks for the performance evaluation in the field of pattern mining. In addition to Apriori and FP-Growth, many other efficient FI mining algorithms have been developed, including *DHP* [19], *H-mine* [22], *Eclat* [29], *COFI-Mine* [6], *negFIN* [2] and *dFIN* [5]. Though conventional FI mining algorithms may perform well on sparse databases, while on dense databases, their performance may be seriously degraded due to a large number of FIs generated during the mining process. Intuitively, the more FIs are generated, the more calculations are needed for the mining process.

3.2 Frequent closed itemset mining

Rather than mining all the FIs, a promising solution is to mine the condensed representations of FIs. Generally speaking, the condensed representation is a small subset of all the FIs. Besides, the total number of itemsets contained in the representation can be much smaller than that of all the FIs. Because the number of itemsets to be found is reduced, the memory usage and execution time required for the mining task are thereby reduced. Different

Table 1 An example of a database

Tid	Transaction
1	ACTW
2	CDW
3	ACTW
4	ACDW
5	ACDTW
6	CDT

kinds of interesting condensed representations of FIs have been proposed, such as *non-derivable itemset* [4], *maximal itemset* [8, 16], *generator itemset* [21], and *closed itemset* [14, 20]. Among these representations, the most popular one is *frequent closed itemset* (abbr. FCI), which is not only condensed but also lossless. In other words, mining only the FCIs will not miss any information of FIs. With correct and efficient deriving algorithms, all the FIs and their supports can be derived from the complete set of FCIs. Due to this interesting property, many studies have focused on FCI mining. Different types of algorithms are developed for efficient mining FCIs, such as *A-Close* [20], *CHARM* [30], *CLOSET* [26], *DCI.Closed* [18] and *NAFCP* [14]. A-Close extends the Apriori algorithm and mainly consists of two stages. In the first stage, it adopts Apriori-like scheme to find some candidates of FCIs. In the second stage, it scans the database again to calculate closure of each candidate and identify all the FCIs from them. CHARM [30] converts the horizontal database into a vertical representation, and uses depth-first search exploration to find FCIs. CLOSET [26] extends the structure of FP-Growth, and uses the recursion and pattern-growth technique to fast generate FCIs from FP-Trees. No matter CHARM or CLOSET, they must keep all the currently found FCIs in memory, which leads them to suffering from large memory consumption and inefficient searching problems. The *DCI.Closed* [18] algorithm is proposed and it alleviates the above problems. It does not need to keep all the currently found FCIs in the memory during the mining process and thereby achieves better performance in generally. Although the above studies have proposed various efficient FCI mining algorithms, none of them have proposed the new algorithms for the deriving task.

3.3 The deriving algorithms

The first deriving algorithm is proposed in the study [21]. In this paper, we call that algorithm as *LevelWise*. LevelWise is a top-down and level-wise algorithm, which takes FCIs as input and outputs the complete FIs and their supports. Its main process is stated as follows. At first, it sorts all the FCIs in descending order of their lengths. Then, each FI of length k ($k \geq 1$) with its support is put into the set L_k . A variable K is set to the length of the longest FCI, and the following iterations are performed.

In each round of the iteration, for each itemset X in L_K , all its subsets of its length $(K-1)$ are generated and put into the set S . For each itemset Y in S , the algorithm searches whether there exists an itemset Y' in L_K such that $Y = Y'$. If this is not the case, the algorithm puts $\langle Y, SP(X) \rangle$ into L_{K-1} . Otherwise, if there exists an itemset Y' in L_K such that $Y = Y'$, then the algorithm compares whether $SP(X)$ is greater than the current support of Y' . If the condition is true, the algorithm replaces the current support of Y' by

$SP(X)$. In contrast, no change is made on the current support of Y' . Finally, the variable K is decreased by 1, and the algorithm processes L_K by the same steps as above and ends the processing when $K = 1$.

Although LevelWise is the first algorithm proposed for the deriving task, it may encounter the following efficiency bottlenecks. When more FIs has to be processed, L_K and L_{K-1} may need to store a considerable number of itemsets. In this case, LevelWise will take a lot of memory and even incurs the problem of memory shortage. Moreover, LevelWise needs to perform a large number of searching and matching operations for the itemsets in L_{K-1} , resulting in low efficiency in execution time. Even if the operation is implemented using a hashing technique, one may encounter the difficulty in designing a good hash function. If the hash function is not well designed, the algorithm may face the serious collision problem and consequently waste a lot of memory spaces.

As surveyed above, only one study [21] had proposed the deriving algorithm. However, it was proposed in 1999, which means that in the past 20 years, no other studies has re-explored this important research topic and developed more efficient and advanced deriving algorithms. In this paper, we address this issue and propose innovative and efficient deriving algorithms, which are presented in details in the next section.

4 The proposed frameworks

4.1 The DFI-list algorithm

This subsection introduces the *DFI-List (Deriving Frequent Itemsets from FCIs based on Cid List)* algorithm. We first introduce the data structure of DFI-List in Section 4.1.1. After that, we introduce the process of DFI-List in detail in Section 4.1.2. In Section 4.1.3, we give a running example to explain how DFI-List works.

4.1.1 Structures of Cid list and SC table

The main storage structure of DFI-List is *Support Count Table* (abbr. *SC Table*) and *Cid List*. During the construction process of SC Table and Cid List, each frequent closed itemset will be endowed with a unique identification number called *Cid (Closed itemset identifier)*. The main purpose of SC Table is to store the Cid and the support count of each FCI. SC Table consists of two fields: Cid and SC (support count). In SC Table, the i -th row stands for the information of the i -th frequent closed itemset X_i . In addition, Cid field is used to store the Cid of X_i , while the SC field is used to store the support count of X_i . For example, Table 2 shows a SC Table. If the Cid of a FCI is c ,

Table 2 The SC Table

Cid	FCI	SC
1	{A}	4
2	{C}	4
3	{AB}	3
4	{AC}	3
5	{CD}	2
6	{ABC}	2

then we can quickly find the support count of the FCI whose Cid is c through the SC Table.

Cid List is a vertical index structure. It consists of two fields: *Itemset* field and *CidSeq* field. Itemset field is used to store a specific itemset X , while CidSeq is used to record the Cid of the super frequent closed itemset of X . Logically, field row in Cid List contains the information about how many FCIs cover a specific itemset. We use $CidSeq(X)$ to represent Cids of all FCIs containing X and we call them as *the CidSeq of the itemset X*. In the initial step, DFI-List will construct an initial Cid List to store each single item and its corresponding CidSeq. Taking Table 3 as an example, the CidSeq of the itemset {B} is {3, 6}, which indicates that the FCIs of Cids 3 and 6, which are {AB} and {ABC}, contain the itemset {B}. In other words, {AB} and {ABC} are two frequent closed itemsets containing the itemset {B}.

The following introduces the algorithm for constructing the SC Table and Cid List in the initial stage of DFI-List. Its pseudo code is show in Fig. 1. Firstly, all the FCIs are sorted in descending order of their support counts (Line 1). When there are less FCIs and all of them can be moved into the memory, we can use some common and fast internal sorting methods to sort FCIs, such as *quick sort*. When there are a large number of FCIs and they cannot be fully moved into the memory, as we can use external sorting methods to sort the FCIs, such as merge sort. After the sorting, a variable Cid is set to 0 (Line 2). Next, DFI-List visits each FCI (Line 3). When visiting to the i -th frequent closed itemset X_i , DFI-List assigns X_i a unique Cid i and updates SC Table and Cid List at the same time (Line 4). In aspect of updating the SC Table, the Cid and SC fields of the i -th row of the SC Table ate set to i and X_i , respectively (Line 5). In aspect of updating the Cid List, Cid i is added to the CidSeq of each

Table 3 The Cid list

Itemset	CidSeq
{A}	{1, 3, 4, 6}
{B}	{3, 6}
{C}	{2, 4, 5, 6}
{D}	{5}

item contained by X_i (Line 6-8). For example, when DFI-List visits the third frequent closed itemset {AB}, DFI-List assigns it a Cid 3. The Cid and SC fields in the third row of the SC Table are set to 3 and $SC(AB) = 3$, respectively. Besides, Cid 3 is added into the CidSeq of the itemset {A} and {B}. After visiting all the FCIs (Line 9), the initial SC Table and Cid List are constructed, which are shown in Tables 2 and 3.

4.1.2 Deriving FIs by DFI-list

DFI-List is a depth-first recursive algorithm. The pseudo code of DFI-List is shown as Fig 2. The input of DFI-List includes a SC Table and an initial Cid List, which are respectively denoted as ST and CL (Line 1). After the process of the algorithm, it outputs all the FIs and their support counts. In the initial stage, $CL = \langle \langle X_1, CidSeq(X_1) \rangle, \langle X_2, CidSeq(X_2) \rangle, \dots, \langle X_n, CidSeq(X_n) \rangle \rangle$, consists of n items and their CidSeqs, where $X_j \in V$ ($1 \leq j \leq n$). In general idea, DFI-List adopts divide-and-conquer strategy. It finds all the FIs related to the itemset I_i first and then find out those FIs not related to I_i but related to I_{i+1} ($1 \leq i \leq n$).

4.1.3 A running example of DFI-list

This subsection gives a running example of the DFI-List algorithm. We take the SC Table in Table 2 and Cid List in Table 3 as examples to illustrate the process of DFI-List. In the initial stage, the Cid List is $CL = \langle \langle A, CidSeq(A) \rangle, \langle B, CidSeq(B) \rangle, \langle C, CidSeq(C) \rangle, \langle D, CidSeq(D) \rangle \rangle$. Since $CidSeq(A) = \{1, 3, 4, 6\}$ and its first Cid is 1, by looking through the SC field of the first row of the SC Table, we can obtain the support count of {A}, which is 4. Similarly, the algorithm uses the same way to process the itemsets {B}, {C}, {D}. After that, we can obtain their support counts, which are respectively 4, 3, 4 and 2.

In the second stage, the internal variable i is set to 1. Then, the algorithm joins the itemsets X_i and X_j ($1 \leq i < j \leq n$) to generate the new itemset $Z = X_i \cup X_j$. Since $i = 1$, $X_i = X_1$ and $X_1 = \{A\}$. The algorithm first joins the itemsets {A} and {B} to generate the itemset {AB}. Then, the algorithm performs the intersection operations on $CidSeq(\{A\}) = \{1, 3, 4, 6\}$ and $CidSeq(\{B\}) = \{3, 6\}$. The result is $CidSeq(\{AB\}) = \{3, 6\}$. Since intersection result is not an empty set, the algorithm looks through the SC field of the third row of the SC Table and obtains the support count of {AB}, which is 3. The algorithm outputs the frequent itemset {AB} with its support count 3. Next, the algorithm joins the itemsets {A} and {C} to generate the itemset {AC}. The $CidSeq(\{AC\})$ is obtained by intersecting $CidSeq(\{A\}) = \{1, 3, 4, 6\}$ and $CidSeq(\{C\}) = \{2, 4, 5, 6\}$, which is $\{4, 6\}$. By looking through the SC field of the fourth row of the SC Table, the support count of {AC} is obtained, which is 3. Then, the

Fig. 1 The algorithm for building initial SC table and Cid list

Algorithm 1 Build SC Table and Cid List Algorithm

```

1: Build(Cid_List, SC_Table)
2: Cid = 0
3: for each frequent closed itemset X do
4:   Cid++
5:   SC_Table[Cid] = SC(X)
6:   for item I in X do
7:     Cid_List[I] = Cid
8:   end for
9: end for

```

frequent itemset {AC} is outputted with its support count 3. The algorithm uses the same way to process {A} and {D}. Because $CidSeq(\{A\}) \cap CidSeq(\{D\}) = \langle 1, 3, 4, 6 \rangle \cap \langle 5 \rangle = \emptyset$, {AD} is not a frequent itemset. DFI-List put the above found frequent itemsets with their CidSeqs into a new Cid List CL_{new} . The result of CL_{new} is $\langle \langle \{AB\}, CidSeq(\{AB\}) \rangle, \langle \{AC\}, CidSeq(\{AC\}) \rangle \rangle$.

Next, the algorithm calls the recursive function, treating CL_{new} as CL , and uses the same way to process CL_{new} . The algorithm first joins the itemsets {AB} and {AC} to generate the itemset {ABC}. Then, the algorithm performs the intersection operations on $CidSeq(\{AB\}) = \langle 3, 6 \rangle$ and $CidSeq(\{AC\}) = \langle 4, 6 \rangle$. The result is $CidSeq(\{ABC\}) = \langle 6 \rangle$. By looking through the SC field of the 6-th row of the SC Table, we can obtain the support count of {ABC}, which is 2.

After processing the itemset {A}, we can obtained all the FIs related to {A}, which are {A}:4, {AB}:3, {AC}:3 and {ABC}:2, where the number beside each itemset is its support count. Next, the internal variable i is increased by one, where the result is 2. Since $i = 2$, $X_i = X_2$ and $X_2 = \{B\}$. By processing the itemset {B} in the same way, we can obtain all the frequent itemsets containing {B} but without containing {A}, which are {B}:3 and {BC}:2. Using the same way to process the itemsets {C} and {D}, we can obtain frequent itemsets {C}:4, {D}:2 and {CD}:2. Since the recursions are all finished and the algorithm has visited

all the itemsets in the initial Cid List, the algorithm is completed. Besides, during the process of the algorithm, all the FIs and their support counts are outputted and obtained.

4.2 The DFI-growth algorithm

In subsection, we introduce the proposed algorithm *DFI-Growth (Deriving Frequent Itemsets from FCIs based on Pattern Growth)*. Since the main internal data structure of DFI-Growth is a *modified FP-Tree* (abbr. *MFP-Tree*), we will briefly introduce the traditional FP-Tree [10] in Section 4.2.1. In Section 4.2.2, we will introduce how we modify the traditional FP-tree to make it be applicable to the deriving tasks. In Section 4.2.3, we will introduce the main processing flow of the DFI-Growth algorithm and present how this algorithm efficiently derives FIs with MFP-Trees.

4.2.1 Briefly introduction to FP-tree

In a FP-Tree, each node N contains five fields, including $N.name$, $N.count$, $N.parent$, $N.hlink$, and $N.child$. The $N.name$ field stores the name of the item J stored in the node N . The $N.count$ field stores the count of J . The $N.parent$ field stores the parent node of N . The $N.hlink$ field is also called *node link*, which points to another node having the same item name as N . By following the node link, all the

Fig. 2 The pseudo code of the DFI-List algorithm

Algorithm 2 The DFI-List Algorithm

```

1: procedure DFI-List( $CL, ST$ )
2:   for each  $CidSeq(X_i)$  in  $CL$  do
3:      $CL_{new} = \phi$ 
4:     for each  $CidSeq(X_j)$  in  $CL$  do
5:        $Z = X_i \cup X_j$ 
6:        $CidSeq(Z) = CidSeq(X_i) \cap CidSeq(X_j)$ 
7:        $c_1 = CidSeq(Z).getFirstCid$ 
8:        $SC(Z) = ST[c_1]$ 
9:       Output :  $Z$  and  $SC(Z)$ 
10:      Add  $CidSeq(Z)$  into  $CL_{new}$ 
11:     end for
12:   end for
13:   DFI-List( $CL_{new}, ST$ )
14: end procedure

```

nodes having the same item name can be easily traversed and obtained. The $N.child$ field stores all the child nodes of N . Each FP-Tree is attached with a Header Table. A Header Table stores the information about items. Let H be a Header Table. In H , each item J in H has three fields: $H[J].name$, $H[J].count$ and $H[J].hlink$. The $H[J].name$ field stores the item name of J . The $H[J].count$ field stores a count related to J . The $H[J].hlink$ field stores a node link, which points to a node having the same item name as J in the FP-Tree. Readers can learn more about information about the traditional FP-Tree and Header Table structures from the reference [10].

4.2.2 Construction of the MFP-tree

The main purpose of DFI-Growth proposed in this work is to efficiently derive the complete FIs with their correct supports from the complete set of FCIs. Its main internal data structure is *modified FP-Tree* (abbr. *MFP-Tree*). The main purpose of the traditional FP-Tree [10] is to mine *frequent itemsets* from the database, instead of deriving FIs from FCIs. Therefore, we modified it to make it be applicable to the deriving task. In the followings, we introduce how to construct a global MFP-Tree to store the information about FCIs.

The algorithm visits each item contained in every FCI and uses the proposed *Maximum Support Selection for Item* (abbr. *MSS-I*) strategy to calculate the correct support count of the items. The MSS-I strategy is introduced below.

Strategy 1 (MSS-I) Let I be an item and $h(I) = X_1, X_2, \dots, X_z$ be the set of all the FCIs containing I , then the support count of I is equal to $\max\{SC(X_i) \mid X_i \in (I)\}$. Equivalently, $SC(I) = \max\{SC(X_1), SC(X_2), \dots, SC(X_z)\}$.

For example, consider the set of FCIs shown in Table 4. The set of all the FCIs containing the item $\{A\}$ is $h(\{A\}) = \{\{CWA\}, \{CWAT\}\}$. Therefore, the support count of $\{A\}$ is $\max\{SC(\{CW\}), SC(\{CWAT\})\} = \max\{4, 3\} = 4$. The Table 5 shows the calculated support count for each item after applying the MSS-I strategy. After applying MSS-I strategy to calculation of support count of items, items are added into a Header Table H and arranged in descending order of their support. Let this order relationship be f . Then, the algorithm sorts items in each FCI by f and inserts the sorted FCI into a global MFP-Tree. A node R is created as the root of a MFP-Tree. Let $X = \{I_1, I_2, \dots, I_k\}$ be the visited sorted FCI, where $I_i \in V$ ($1 \leq i \leq k$) and X consists of k distinct items. Next, the algorithm calls the subroutine $InsertFCI(R, I_1)$ to insert X into the MFP-Tree.

The pseudo code of the subroutine $InsertFCI(N, I_t)$ is presented in Fig. 3, which works as follows. The algorithm first checks whether the input node N has a child node N' such that $N.name$ is equal to $N'.name$ (Line 1). If the

Table 4 FCI and their support count

Frequent closed itemset	Support count
{CWA}	4
{CW}	5
{CD}	4
{C}	6
{CT}	4
{CWD}	3
{CWAT}	3

condition is true, the algorithm uses the proposed *Maximum Support Replacement* (abbr. *MSR*) strategy to set $N'.count$ (Line 2). The algorithm will choose the largest value from $N'.count$ and $SC(X)$ as the value of $N'.count$.

Otherwise, if N does not exist a child node N' such that $N.name$ is equal to $N'.name$, the algorithm creates a new node N' as a child node of N (Line 4) and sets $N'.count$ to $SC(X)$ (Line 5). Next, the variable t is increased by 1 (Line 8). If $t \leq k$, the algorithm recursively calls $InsertFCI(N', I_t)$ to insert the t -th item of X into the tree (Line 9).

Strategy 2 (MSR) If $SC(X) > N'.count$, then replacing $N'.count$ by $SC(X)$.

Figure 4 shows an example for inserting FCIs using the MSR strategy. The first FCI to be inserted is $\{CWA\}:4$, where the number beside the itemset is its support count. Because there are no nodes under the root, a path of nodes is created and the counts of each created node are set to 4. The result is shown in Fig. 4a. The second FCI to be inserted is $\{CW\}:5$. The first item of $\{CW\}$ is $I_1 = \{C\}$. The algorithm calls $InsertFCI(R, \{C\})$ to process the first item of $\{CW\}$, where R is the root of the MFP-Tree. The node N is set to R and the variable t is 1. Because there exists a node $N' = \{C\}$ under the root R of the MFP-Tree and $I_1 = \{C\}$ is equal to $N'.name$, the algorithm applies the MSR strategy to setting $N'.count$. Since $SC(\{CW\})$ (i.e., 5) $\geq N'.count$ (i.e., 4), $N'.count$ is replaced by $SC(\{CW\}) = 5$. The variable t is increased by 1.

Then, the algorithm recursively calls $InsertFCI(N', \{W\})$ to process $I_2 = \{W\}$ (i.e., the second item of $\{CW\}$), where N' currently is the first node $\{C\}$ under the root R in Fig. 4(b). The node N is set to the node $\{C\}$ and the variable t is 2. Because N has a child node $N' = \{W\}$ such that $N'.name$ is equal to I_2 , $N'.count$ is set by applying the MSR strategy. Since $SC(\{CW\})$ (i.e., 5) $\geq N'.count$ (i.e., 4), $N'.count$ is replaced by 5. After inserting the second frequent closed itemset $\{CW\}$, the MFP-Tree is shown as Fig. 4b. Figure 4c and d show the MFP-Trees after inserting the third and fourth frequent closed itemsets (i.e., $\{CD\}:4$ and $\{C\}:6$), respectively.

Table 5 An example of the MSS-I strategy

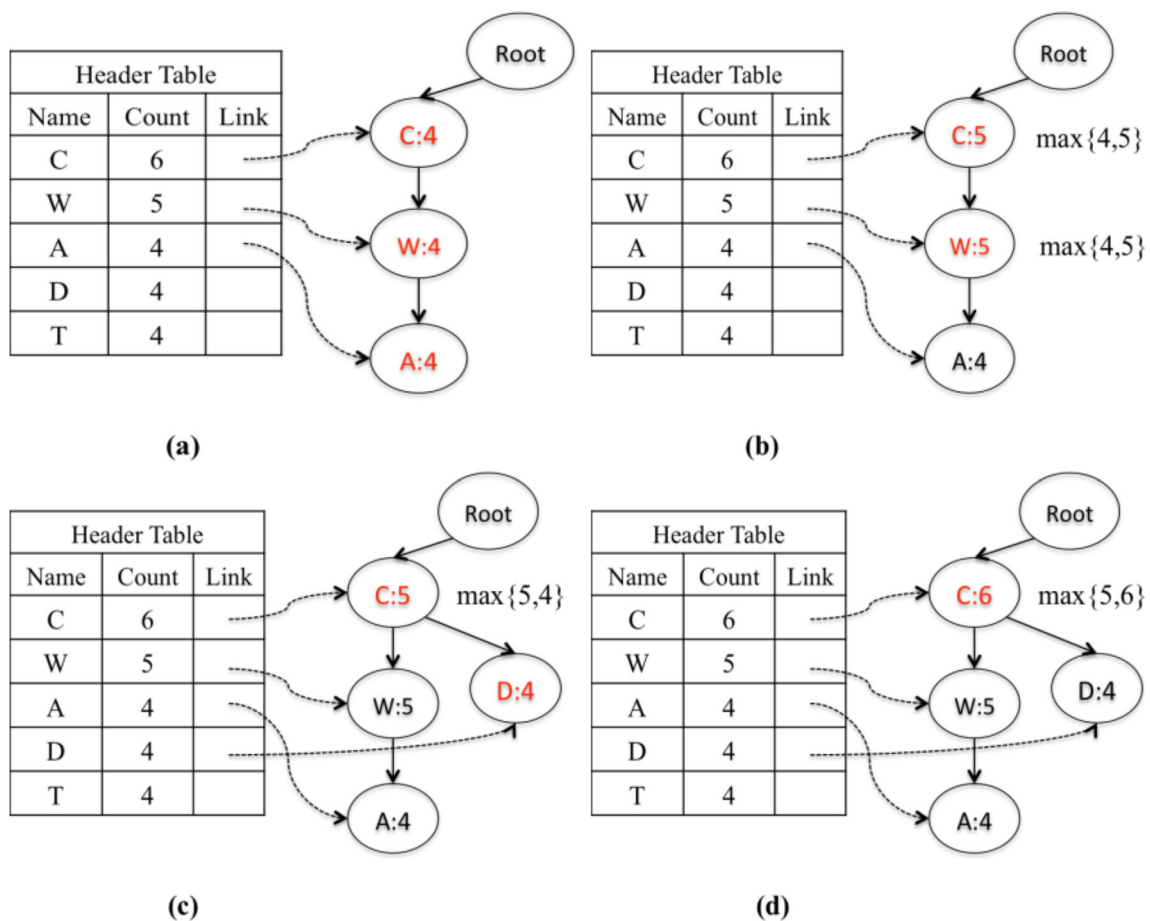
Itemset	Maximum support selection	Support count
{C}	$\max\{SC(\{CWA\}), SC(\{CW\}), SC(\{CD\}), SC(\{C\}), SC(\{CT\}), SC(\{CWD\}), SC(\{CWAT\})\}$	6
{W}	$\max\{SC(\{CWA\}), SC(\{CW\}), SC(\{CWD\}), SC(\{CWAT\})\}$	5
{A}	$\max\{SC(\{CWD\}), SC(\{CWAT\})\}$	4
{D}	$\max\{SC(\{CD\}), SC(\{CWD\})\}$	4
{T}	$\max\{SC(\{CT\}), SC(\{CWAT\})\}$	4

Fig. 3 The pseudo code of the *InsertFCI* subroutine**Algorithm 3** The pseudo code of the *InsertFCI* subroutine

```

1: if ( $N$  has a child  $N'$  such that  $N'.name == I_t$ ) then
2:    $N'.count = \max\{N'.count, SC(X)\}$ 
3: else
4:   Create a new child node  $N'$  and
5:   Set  $N'.name = I_t, N'.count = SC(X), N'.parent = N$ 
6:   Set  $N'.link$  to the lastly created node having the same item name as  $I_t$ 
7: end if
8:  $t = t + 1$ 
9: if ( $t \leq k$ ) then, call InsertFCI( $N', I_t$ )
10: end if

```

**Fig. 4** An example of the MSR strategy. **a** Insert CWA: 4. **b** Insert CW:5. **c** Insert CD: 4. **d** Insert C: 6

During the building process of the global MFP-Tree, all the tree nodes having the same item name will be linked together using the node links of the Header Table and related nodes in MFP-Tree. By following the node links, nodes having the same item name can be efficiently traversed by DFI-Growth.

4.2.3 Deriving FIs by DFI-growth

After inserting all the sorted FCIs into the MFP-Tree, a global MFP-Tree T with a Header Table H are constructed. Then, the algorithm calls the recursive function $DFI-Growth(T, H, \alpha)$ to derive FIs with their support counts by using T and H , where α is a *prefix itemset* and currently it is initialized to $\emptyset$. In the followings, we introduce how the recursive function $DFI-Growth(T, H, \alpha)$ works, which mainly consists of six main steps.

Step 1 The algorithm visits each item in the Header Table H in a reverse order of f . By following the node link of $H[J].hlink$, the algorithm efficiently finds all the nodes having the same item name as J from the MFP-Tree T .

Step 2 Let $N(J) = \langle N_1, N_2, \dots, N_m \rangle$ be the sequence of found nodes having the item name as J . For each node $N_i \in N(J)$ ($1 \leq i \leq m$), the algorithm traces all of its ancestor nodes to the root node R . The algorithm collects the item names of all the traced nodes in order, and forms a special sequence P_i ($1 \leq i \leq m$) called *conditional pattern*. Each conditional itemset is attached with a count called the *count of the conditional pattern*. The *count of the conditional pattern* is denoted as $SC(P_i)$, which is set to $N_i.count$ initially. After processing all the nodes in $N(J)$, a conditional pattern base of the prefix itemset $(\{\alpha \cup J\})$ is constructed. We use the notation $CPB(\{\alpha \cup J\}) = \{P_1:SC(P_1), P_2:SC(P_2), \dots, P_m:SC(P_m)\}$ to represent the conditional pattern base of the prefix itemset $\{\alpha \cup J\}$.

Step 3 The algorithm scans the conditional pattern base $CPB(\{\alpha \cup J\})$ once, and collects every single items in each conditional pattern into a set Z . For each item $I \in Z$, the algorithm derives a frequent itemset $\{\alpha \cup J \cup I\}$ and calculates its support count. The support count of $\{\alpha \cup J \cup I\}$ is calculated by applying the proposed *Maximum Support Selection for ItemSet* (abbr. *MSS-S*) strategy.

Strategy 3 (MSS-S) Let I be an item appearing in $CPB(\{\alpha \cup J\})$ and $I \in Z$, $P(I) = \{P_1':SC(P_1'), P_2':SC(P_2'), \dots, P_n':SC(P_n')\}$ be the set of all the conditional patterns containing I in $CPB(\{\alpha \cup J\})$. The support count of the frequent itemset $\{\alpha \cup J \cup I\}$ is equal to $\max SC(P_i')$

$| SC(P_i') \in CPB(\{\alpha \cup J\})$ ($1 \leq i \leq n$). Equivalently, $SC(\{\alpha \cup J \cup I\}) = \max SC(P_1'), SC(P_2'), \dots, SC(P_n')$.

For example, Table 6 gives an example of the conditional pattern base of $\{T\}$, in which there are two conditional patterns $\{CWA\}:3$ and $\{C\}:4$. Currently, the prefix itemset α is $\emptyset$. The algorithm scans the conditional pattern base $CPB(\{T\})$ once and collects every single item into the set Z . After that, the set $Z = \{C, W, A\}$. By applying the *MSS-S* strategy, the support count of $\{\emptyset \cup T \cup C = \{TC\}$ is $\max\{SC(\{CWA\}), \max SC(\{C\})\} = \max\{3, 4\} = 4$. The algorithm calculates the support count of $\{TW\}$ and $\{TA\}$ by the same strategy. The result is shown in Table 7. Therefore, the algorithm finds three frequent itemsets with their support counts from $CPB(\{T\})$, which are $\{TC\}:4$, $\{TW\}:3$ and $\{TA\}:3$.

Step 4 For each item $I \in Z$, the algorithm adds it into a new header table H' in the order f . Then, the algorithm sorts items in each conditional pattern in $CPB(\{\alpha \cup J\})$ according to f .

Step 5 The algorithm constructs a conditional MFP-Tree T' for the prefix itemset $(\{\alpha \cup J\})$, which works as follows. A new node R' is created. Then, the algorithm inserts each sorted conditional pattern in $CPB(\{\alpha \cup J\})$ into T' . For each sorted conditional pattern Y , the algorithm calls the function $Insert-FCI(R', I_1)$ to process it, where R' is the root of the conditional MFP-Tree T' and I_1 is the first item in Y . After constructing the conditional MFP-Tree T' , all the nodes in T' having the same item name are linked together by the node links.

Step 6 Excepting the root node R' , if the constructed MFP-Tree T' has other nodes, the algorithm calls the subroutine $DFI-Growth(T', H', \alpha \cup J)$ to recursively derive FIs.

5 Experimental evaluation

In this section, we evaluate the performance of the proposed algorithms DFI-Growth and DFI-List. We compare them with the state-of-the-art algorithms LevelWise [21] and dFIN [5]. The purpose of LevelWise, DFI-Growth and DFI-List is the same, that is, to derive FIs with their correct supports from FCIs. Different from the above three algorithms, the purpose of dFIN is to efficiently mine FIs in datasets. The computer used for the experiments

Table 6 The conditional pattern base of $\{T\}$

Conditional pattern	Count
$\{CWA\}$	3
$\{C\}$	4

Table 7 Calculating support count of FI by using the MSS-S strategy

Frequent itemset	Maximum support selection	Support count
{TC}	$\max\{SC(\{CWA\}), SC(\{C\})\}$	4
{TW}	$\max\{SC(\{CWA\})\}$	3
{TA}	$\max\{SC(\{CWA\})\}$	3

is equipped with Intel® Core™ i7-8700 3.20GHz, 16G memory, Windows10 Premium OS. All the experiments are performed in commended line mode. All the compared algorithms are implemented in Java programming language. Both execution time and memory usage of the algorithms are evaluated. We measure the maximum memory usage of the algorithms by using the Java API. Six real datasets are used for the performance evaluation. The real datasets include Mushrooms, Chess, Connect, BMS_WebView_1, Retail and ChainStore. All the datasets are downloaded from SPMF [7]. Table 8 shows characteristics of the different datasets. The Mushrooms, Chess and Connect datasets are dense datasets. The Mushrooms dataset records the growth of different types of Mushrooms. The Chess dataset records steps of chess and the Connect dataset records the game connection history. Connect contains generally longer transactions than Chess. The BMS_WebView_1, Retail and ChainStore are sparse datasets. BMS_WebView_1 contains some very long transactions. Retail contains more transactions and items than BMS_WebView_1. Chainstore is the largest dataset and it contains 1,112,949 transactions and 46,086 different kinds of items. Table 8 summarizes the characteristics of the above datasets, In Table 8, D is the number of transactions, V is the total number of distinct items, AL is the average length of transactions. For scalability test, a synthetic dataset named D500KV2K is generated by the data generator provided by SPMF [7] (Figs. 5 and 6).

5.1 Experiments on dense dataset

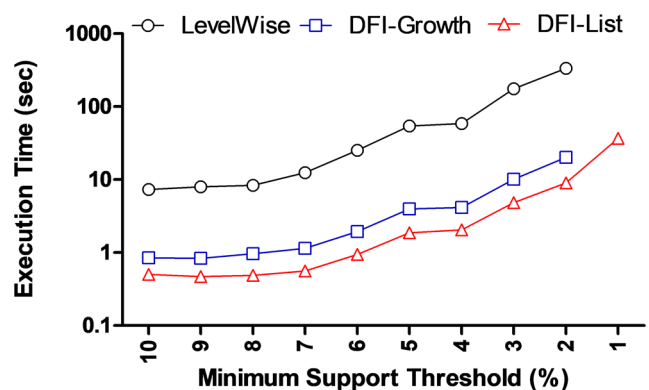
In this subsection, we evaluate the efficiency of the algorithms on different dense datasets. We set different

Table 8 Characteristics of datasets

Dataset	D	V	AL	Type
Mushrooms	8,124	119	23	Dense
Chess	3,196	75	37	Dense
Connect	67,557	129	43	Dense
BMS_WebView_1	59,602	497	2.51	Sparse
Retail	88,162	16,470	10.30	Sparse
Chainstore	1,112,949	46,086	7.23	Sparse
D500KV2K	500,000	2,000	20	Synthetic

minimum support thresholds and find the FCIs from these datasets. Figures 5 and 6 respectively show the execution time and memory usage of different algorithms under different thresholds. Figures 7, 8 and Table 9 show the number of FCIs and FIs contained in the Mushrooms dataset under different thresholds. On the Mushrooms dataset, when the minimum support threshold is greater than 2%, DFI-List has the best performance in terms of execution speed, which is about 10 times faster than LevelWise. In terms of memory usage, when the minimum support threshold is greater than 4%, DFI-List generally consumes less memory than LevelWise. When the minimum support threshold is less than 2%, neither LevelWise nor DFI-Growth can continue to run due to running out of memory. While DFI-List can continue to process and it only uses a very small amount of memory. On Mushrooms dataset, DFI-List has the best performance. This is because LevelWise needs to keep FIs with lengths k and $(k-1)$ at the same time. Therefore, when there are a large number of FIs, it has exhausted the memory and cannot restore all FIs completely. The memory usage of DFI-List is mainly used for recursive enumeration of FIs. Because there are fewer items of dense data, the number of itemsets that DFI-List needs to enumerate is also small, so it has a relatively low memory usage.

Figures 9, 10, 11 and 12 show the experimental results on Chess dataset. Table 10 shows the number of FCIs and FIs contained in Chess dataset. As shown in Fig. 9, When the minimum support threshold is set to 40%, DFI-Growth can derive all the FIs in only 25 seconds, while LevelWise takes more than 36 hours ($36\text{hr} \times 60\text{mins} \times 60\text{sec} = 129,600$ seconds). Because the execution time of DFI-Growth and DFI-List is significantly faster than that of LevelWise and the performance gap is too big, in the Fig. 9 we do not show the execution time of LevelWise when the threshold is less than 40%. This is because when the threshold is below 40%, there are many FIs in the Chess dataset, LevelWise needs to keep all the FIs of length k and $(k-1)$ at the same time during its process. Therefore, when the numbers of FIs of lengths k and $(k-1)$ are too large, LevelWise may take a lot of

**Fig. 5** Execution time on mushrooms dataset

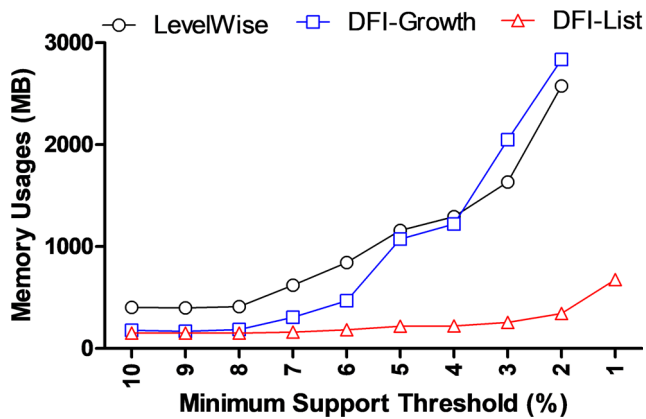


Fig. 6 Memory usage on mushrooms dataset

time on searching and comparison, resulting in poor execution speed. For DFI-Growth, because there are fewer items in this dataset, FCIs can be well compressed into the MFP-Tree. In addition, DFI-Growth uses a depth-first and divide-and-conquer strategy to quickly generate FIs from the compressed MFP-Tree. Therefore, it can achieve better performance than LevelWise. As shown in Figs. 9 and 10, When the threshold is less than 40%, both LevelWise and DFI-Growth run out of memory and cannot perform. However, DFI-List still performs efficiently (Fig. 13 and 14).

Next, we test the performance of the algorithms on the Connect dataset. Compared with the Mushrooms and Chess datasets, the Connect dataset contains more types of items and longer transactions, so the number of FCIs is also larger. Figures 13 and 14 show the execution time and memory usage of the different algorithms on this dataset. Figures 15, 16 and Table 11 show the numbers of FCIs and FIs contained in Connect under different thresholds. On the Connect dataset, when the threshold is greater than 60%, the execution speed of DFI-Growth and DFI-List is better than LevelWise. Besides, the execution time of LevelWise is the slowest. Moreover, DFI-Growth and DFI-List are 26 times faster than LevelWise, and they only takes 23 seconds to derive all the FIs.

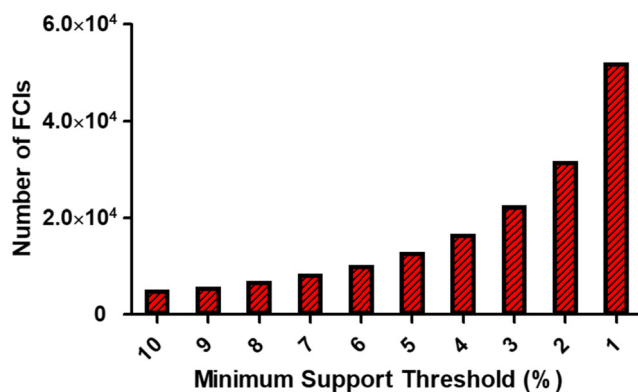


Fig. 7 Number of FCIs on mushrooms dataset

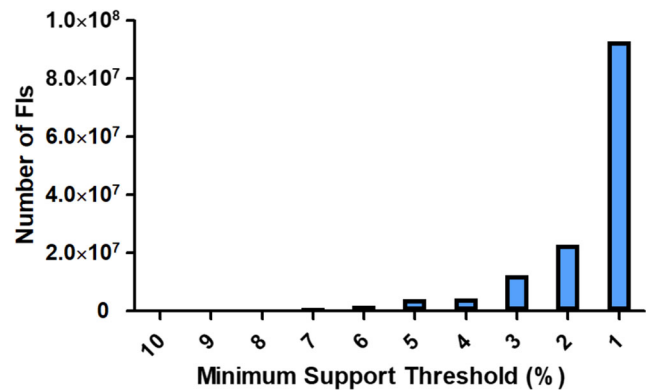


Fig. 8 Number of FIs on mushrooms dataset

However, when the threshold is below 60%, both LevelWise and DFI-Growth fail to execute because of memory exhaustion. As shown in Figs. 13 and 14, DFI-List still perform well and it only uses much less memory as compared with LevelWise and DFI-Growth.

5.2 Experiments on sparse dataset

First, we test the efficiency of all the compared deriving algorithms on the BMS_WebView_1 dataset. Figures 17 and 18 respectively show the execution time and memory usage of the different algorithms under varied thresholds. Figures 19, 20 and Table 12 show the number of FCIs and FIs contained in BMS_WebView_1 under varied thresholds. On the BMS_WebView_1 dataset, when the minimum support threshold is higher than 0.1%, all the compared algorithms perform well. As the minimum support threshold decreases, the performance gap between LevelWise and the other two algorithms is also increased. For example, when the minimum support is set to 0.06%, DFI-Growth and DFI-List can derive all the FIs within 2 seconds, but LevelWise takes more than 172 seconds. DFI-Growth executes approximately 86 times faster than LevelWise. This is because BMS_WebView_1 is a sparse dataset, when the

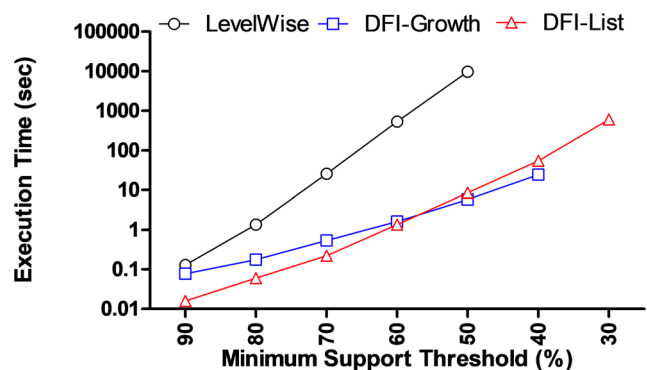


Fig. 9 Execution time on chess dataset

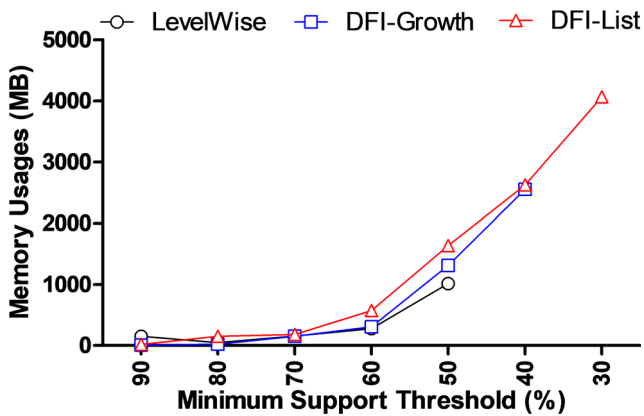


Fig. 10 Memory usage on chess dataset

threshold is high, the lengths and number of FCIs are relatively small, and there is not much difference in execution time. However, when the threshold is less than 0.06%, longer frequent closed itemsets begin to be produced. Therefore, when the threshold is below 0.06%, the difference in execution speed between the two algorithms becomes large. In terms of memory usage, as shown in Fig. 18, when the threshold is high, there is not much difference in memory usage between LevelWise and DFI-Growth. However, as the minimum support threshold decreases, the performance gap between LevelWise and our proposed algorithms increases obviously. On the BMS_Webview_1 dataset, the memory usage of DFI-List is the highest. This is because DFI-List adopts recursive enumeration for deriving the FIs. When there are many items in the sparse datasets, DFI-List needs to enumerate a lot of itemsets. Therefore, it may spend more memory usage (Figs. 19 and 20).

Next, we test the efficiency of the different algorithms on the Retail dataset. Figures 21, 22, 23 and 24 show the performance of the different algorithms on the Retail dataset. Table 13 shows the number of FCIs and FIs contained in Retail dataset. Figures 23 and 24 show the number of FCIs

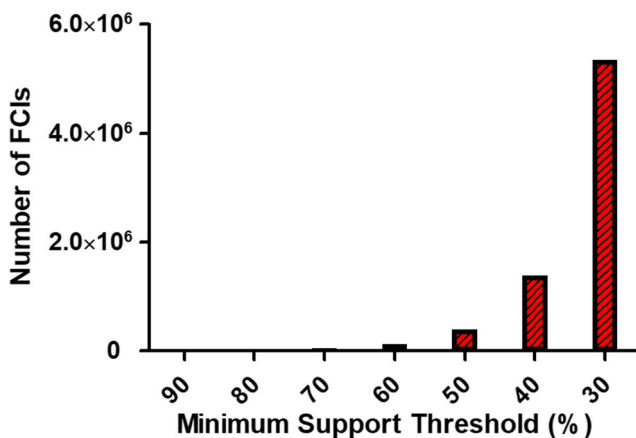


Fig. 11 Number of FCIs on chess dataset

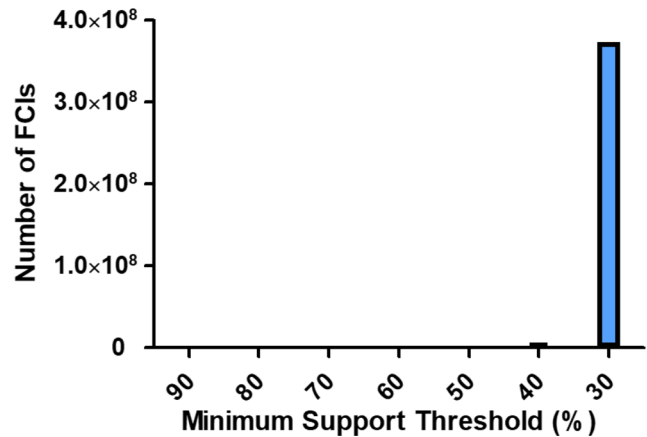


Fig. 12 Number of FIs on chess dataset

and FIs contained in Retail under different thresholds. As shown in Figs. 21 and 22, DFI-List has the fastest execution speed and the highest memory usage on the Retail dataset under any settings of the threshold. As shown in Fig. 21, although the performance of all the compared algorithms is good when the threshold is high, as the threshold is reduced to 0.01%, the execution time of LevelWise increases a lot. For example, when the threshold is set to 0.01%, DFI-Growth can finish the task in 50 seconds, while LevelWise takes 1,855 seconds. Figure 22 shows the memory usage of the different algorithms under different thresholds. When the threshold is set to 0.01%, DFI-List fails to complete the deriving task due to memory exhaustion. This is because DFI-List relies on CidSeqs of items to enumerate FIs. When there are many distinct items, DFI-List may produce a lot of CidSeqs during the recursive exploration process.

Figures 25, 26, 27 and 28 show the performance of the different algorithms on the ChainStore dataset. Table 14 shows the number of FCIs and FIs contained in ChainStore dataset. ChainStore is a real transaction dataset from American chain stores, which contains 1,112,949 transactions and 46,086 items, and it is a very large dataset. Figures 27 and 28 show the numbers of FCIs and FIs in this dataset under different thresholds. As shown in

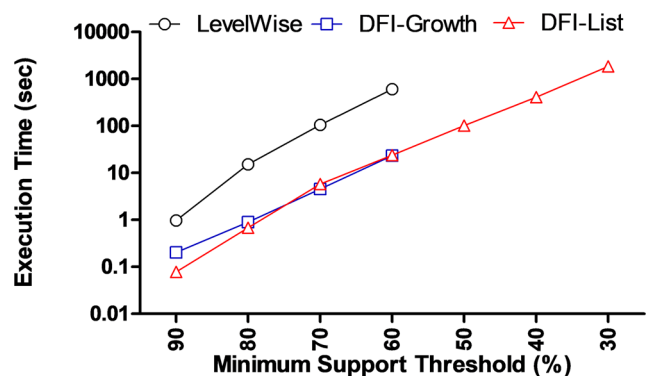


Fig. 13 Execution time on connect dataset

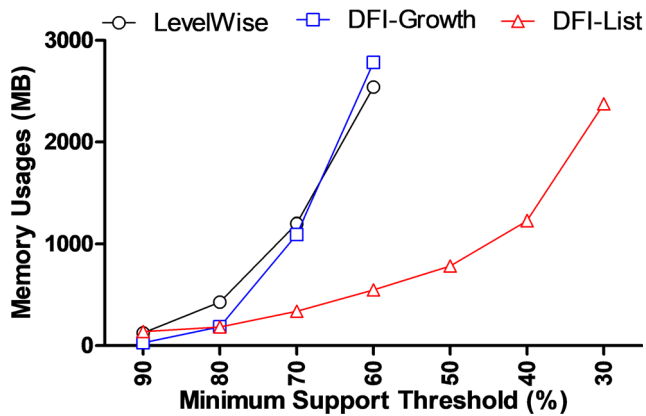


Fig. 14 Memory usage on connect dataset

Fig. 25, DFI-List has the best execution speed. As shown in Fig. 26, the memory usage of LevelWise and DFI-Growth are competitive. But the memory usage of DFI-List is much more than that of LevelWise and DFI-Growth. Comparing with DFI-Growth and LevelWise, DFI-Growth has the lowest memory efficiency on this dataset.

5.3 Comparison with FIM algorithm and scalability test

In this subsection, we compare the performance of the advanced FIM algorithm dFIN [5] with the proposed algorithms DFI-Growth and DFI-List. Notice that the input data of the three algorithms are different, where the input of dFIN is a transaction dataset, while the input of DFI-Growth and DFI-List is the complete set of FCIs with their supports. The output of the three algorithms is the same, which is the complete set of FIs with their supports. To reveal the interesting properties of the three algorithms, we choose the three datasets carrying with different properties for the evaluation, which are Mushrooms, Chainstore and D500KV2K. Mushrooms is a small and real-life dense dataset containing few frequent items. Chainstore is a real-life dataset containing more than one million transactions.

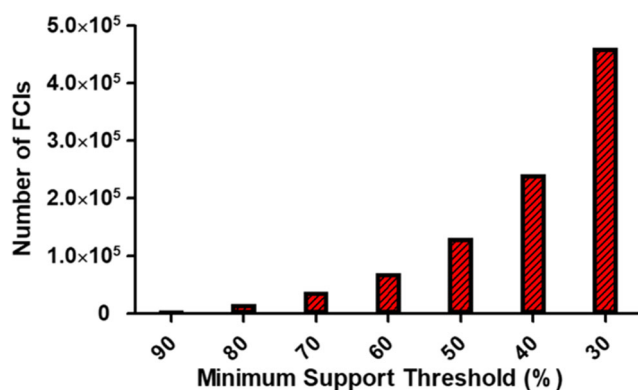


Fig. 15 Number of FCIs on connect dataset

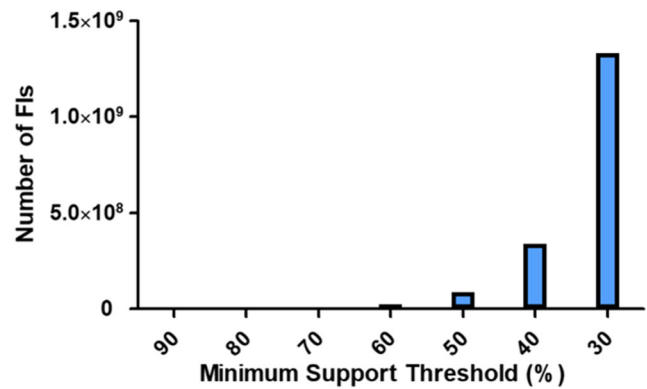


Fig. 16 Number of FIs on connect dataset

D500KV2K is a synthetic dataset generated by data generator of SPMF and it is used for testing the scalability of the compared algorithms.

Figures 29 and 30 show the execution time and memory usage of the compared algorithms on Mushrooms. As shown in Fig. 29, in general dFIN has the fastest execution time. This is because Mushrooms is a small dense dataset, which consists of only 8,124 transactions (as shown in Table 8), and there are relatively small differences in consisted items in different transactions. In this case, the internal data structure adopted by dFIN allows to compressing the database very well and effectively reducing the computational cost for calculating supports of itemsets. Different from dFIN, the input of DFI-Growth and DFI-List is the set of FCIs. From Fig. 7 and Table 9, it can be seen that the number of FCIs is much larger than the number of transactions of Mushrooms (i.e., 8,124). This means that DFI-Growth and DFI-List require a higher search and memory cost when deriving FIs with their supports (Table 10). Figure 30 shows the memory usage of the three algorithms. Because the data structure of dFIN is able to well compress the database and the database is small, the memory usage is less. As for DFI-Growth and DFI-List, they compress FCIs into their data structures, and the number of FCI is much larger than the number of

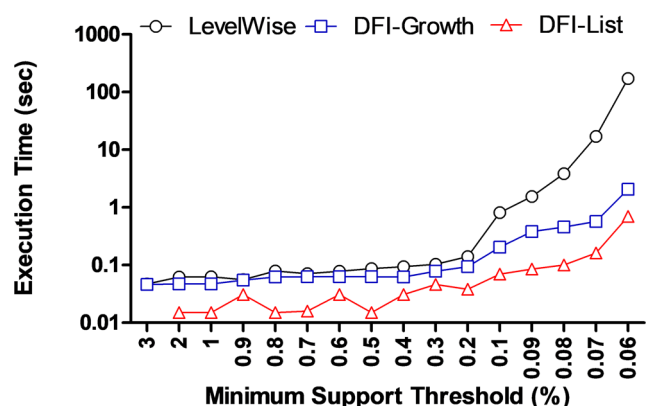


Fig. 17 Execution time on BMS_WebView_1 dataset

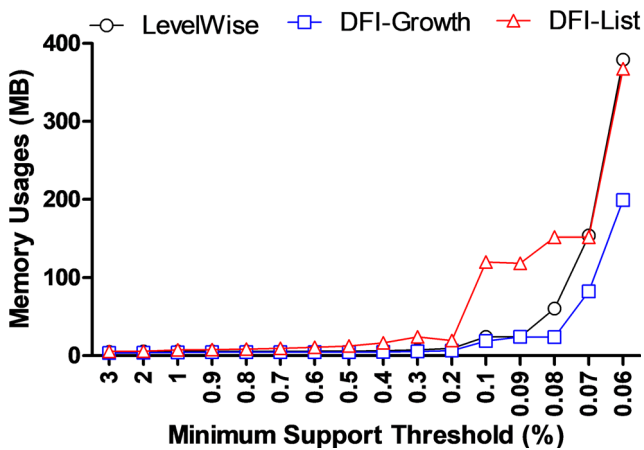


Fig. 18 Memory usage on BMS_WebView_1 dataset

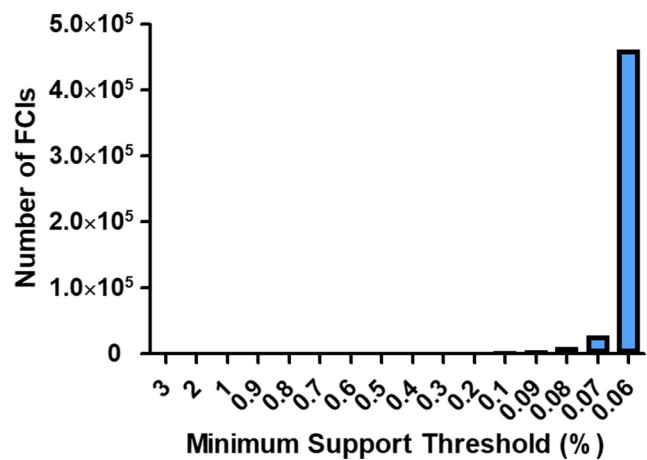


Fig. 20 Number of FIs on BMS_WebView_1 dataset

transactions of Mushrooms. Therefore, they will consume a higher amount of memory usage. Although the performance of dFIN in this experimental setting is better, if the user retains the FCIs but does not retain the original database, the user cannot use dFIN to discover FIs. However, DFI-Growth and DFI-List can be a good solution for finding out the complete information of FIs.

Figures 31 and 32 respectively show the execution time and memory usage of the compared algorithms on Chainstore. As shown in Fig. 31, the execution efficiency of DFI-Growth and DFI-List is much better than dFIN. This is because Chainstore is a dataset with more than one million transactions. In this case, performing FIM by dFIN involves a large amount of calculation and space cost, including: repeatedly scanning the entire original database, compressing the database into the data structure of dFIN, and generating each FI from the structure, and calculating

supports. As shown in Table 11 and Fig. 27, the number of FCIs is much smaller than that of transactions in Chainstore. Therefore, DFI-Growth and DFI-List can restore all the FIs from a small amount of FCIs in a very efficient manner. Figure 32 shows the memory usage of the algorithms in this experiment. As shown in Fig. 32, DFI-Growth has the lowest memory usage. This is because the internal storage structure of DFI-Growth can compress FCIs very well. DFI-List has the highest memory usage. This is because DFI-List enumerates FI in a depth-first and recursive manner. When there are many frequent items, a large number of Cid-Seqs needs to be constructed during the deriving process of DFI-List, leading to a high memory usage (Tables 12, 13 and 14).

This experimental result coincides with the third application scenario mentioned in the Introduction Section. Although dFIN can be used to mine FIs, it must re-scan the original database and calculate the support for each FI. When the processed dataset is very large, this may result in a

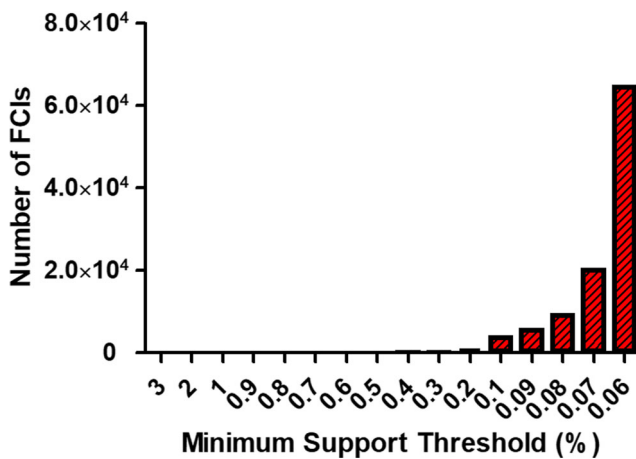


Fig. 19 Number of FCIs on BMS_WebView_1 dataset

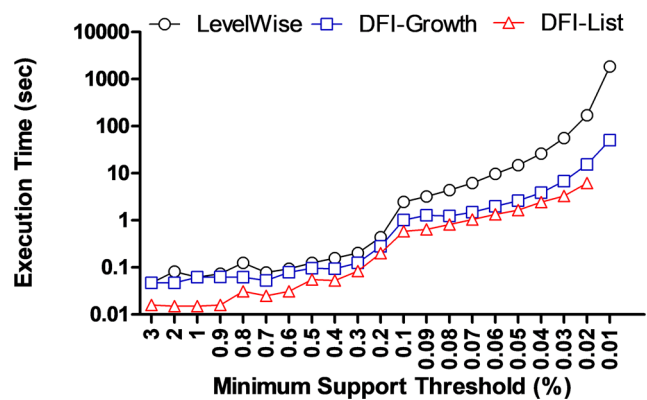


Fig. 21 Execution time on retail dataset

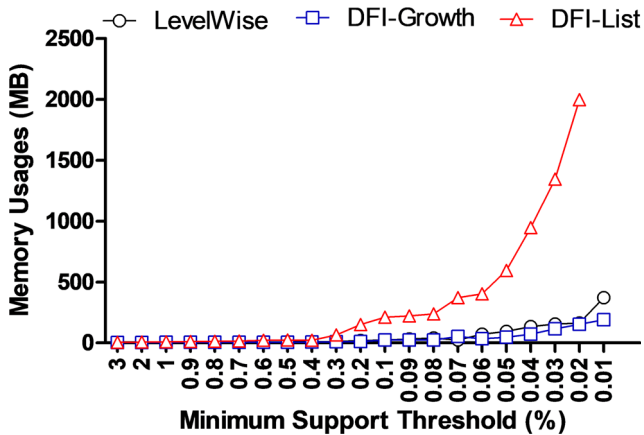


Fig. 22 Memory usage on retail dataset

large amount of computational cost. However, if the number of FCIs is small, the proposed DFI-Growth or DFI-List can work very well and the complete FIs can be efficiently derived without any scanning of the original database, thus saving a lot of time and processing cost.

We divide the D500KV2K dataset into the other four smaller datasets, where the transaction numbers of these datasets are 100K, 200K, 300K, 400K and 500K. In this experiment, the minimum support is fixed to 0.01%. Figures 33 and 34 respectively show the execution time and memory usage of the different algorithms on these datasets. As shown in Figs. 33 and 34, the execution speed and memory usage of dFIN has increased significantly as the number of transactions increases. However, both DFI-Growth and DFI-List have good scalability in this experimental setting. This is because when the minimum support is set to 0.01%, the number of FCIs is about 2,000, so DFI-Growth and DFI-List have quite good performance. For example, both DFI-Growth and DFI-List completes the deriving task within 1 second. This experimental result

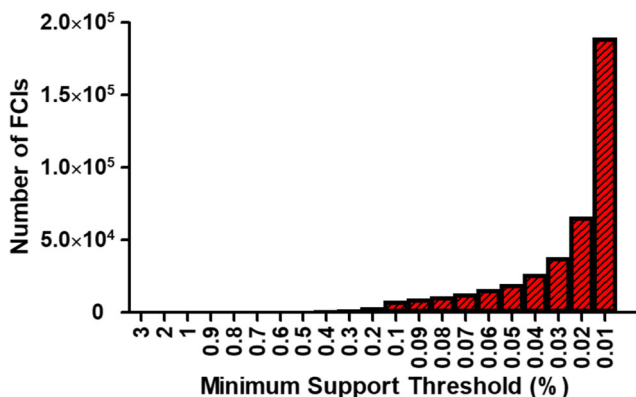


Fig. 23 Number of FCIs on retail dataset

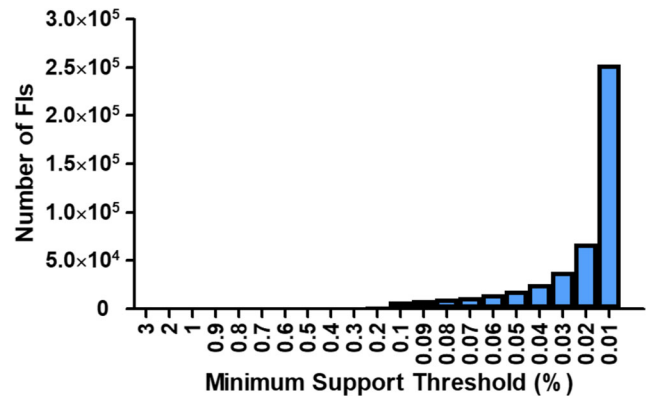


Fig. 24 Number of FIs on retail dataset

highlights the application timing and main advantages of the proposed deriving algorithms.

5.4 Summary of the experimental results

From the above experimental results, we can observe that DFI-Growth is particularly suitable for processing FCIs in sparse datasets. The internal storage structure of DFI-Growth is MFP-Tree. When processing FCIs in dense datasets, it allows to effectively compressing FCIs into the MFP-Tree and thereby a lot of memory can be saved. DFI-Growth is a depth-first and divide-and-conquer algorithm, and it uses pattern-growth technique to generate FIs. Therefore, it will not generate any candidates during the process, resulting in the benefits of fast searching and saving memory. However, the nodes of the MFP-Tree need to store a lot of information, and many MFP-Trees and Header Tables will be created during the recursive process. Therefore, the memory usage often increases significantly as the threshold is decreased. Even so, DFI-Growth still uses less memory than LevelWise.

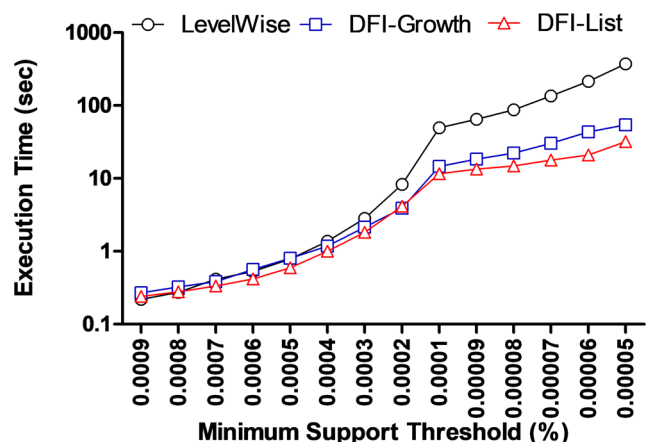


Fig. 25 Execution time on chainstore dataset

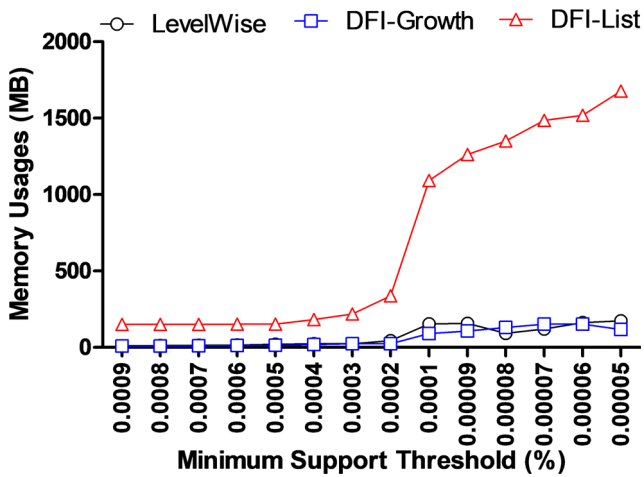


Fig. 26 Memory usage on chainstore dataset

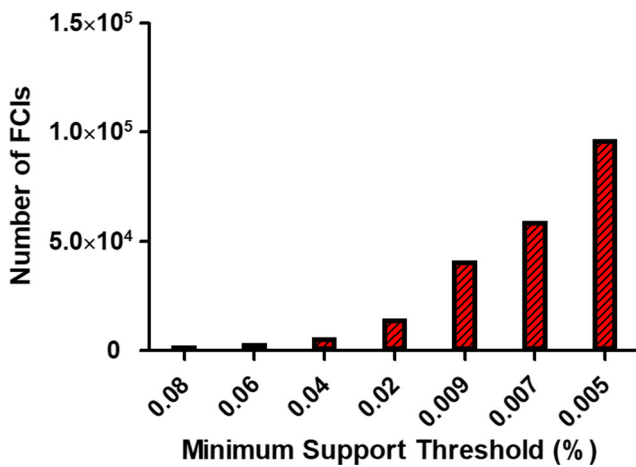


Fig. 27 Numbers of FCIs on chainstore dataset

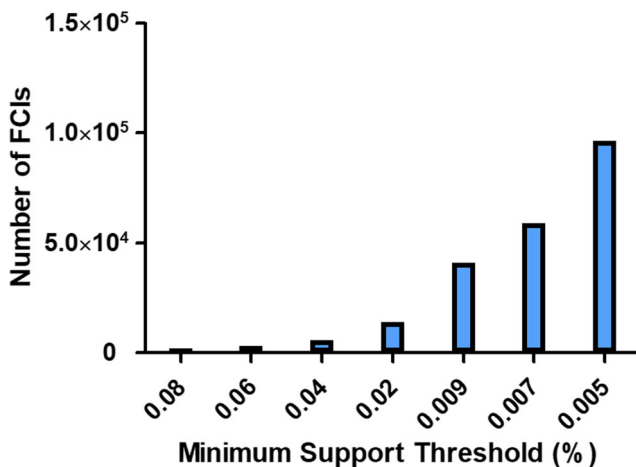


Fig. 28 Number of FIs on chainstore dataset

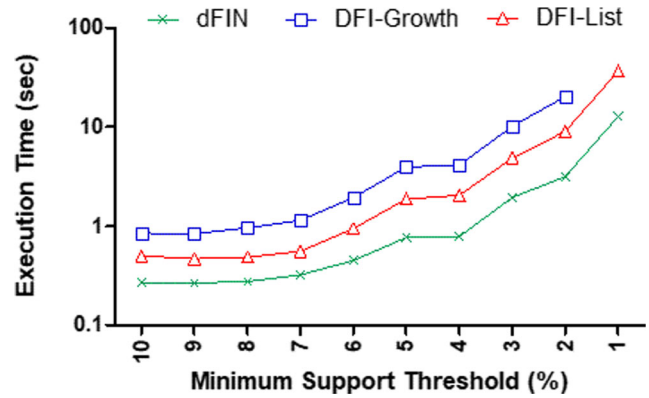


Fig. 29 Execution time comparison on mushrooms

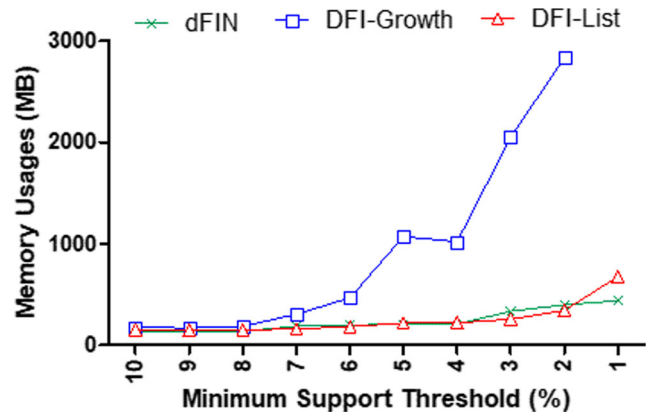


Fig. 30 Memory usage comparison on mushrooms

Table 9 Number of FIs and FCIs on mushrooms dataset

Minimum Support Threshold(%)	Minimum Support Count	Number of frequent Itemsets	Number of frequent Closed Itemsets
10	841	600,817	4,850
9	757	618,189	5,571
8	673	650,003	6,683
7	589	970,105	8,202
6	504	1,843,327	10,037
5	420	4,137,547	12,789
4	336	4,345,935	16,408

Table 10 Number of FIs and FCIs on chess dataset

Minimum Support Threshold(%)	Minimum Support Count	Number of Frequent Itemsets	Number of Frequent Closed Itemsets
90	2,876.4	622	498
80	2,556.8	8,227	5,083
70	2,237.2	48,731	23,892
60	1,917.6	254,944	98,392
50	1,598	1,272,932	369,450
40	1,278.4	6,439,702	1,361,157

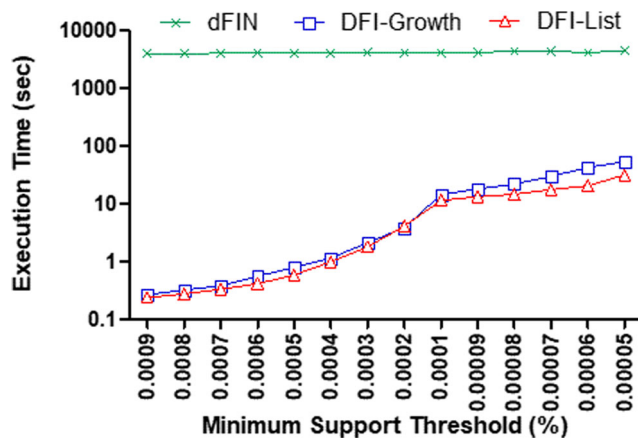


Fig. 31 Execution time comparison on chainstore

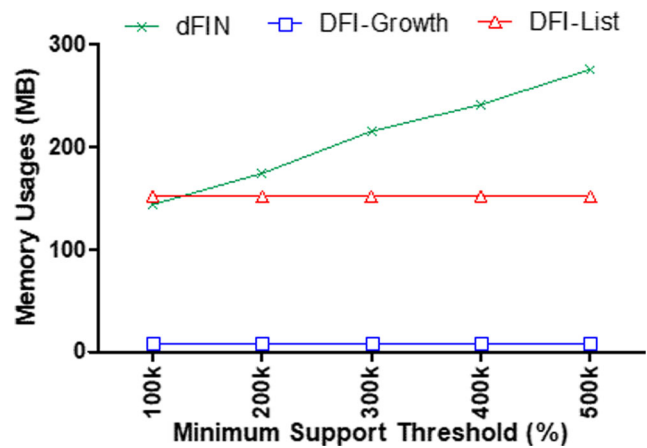


Fig. 34 Scalability test on D500KV2K dataset

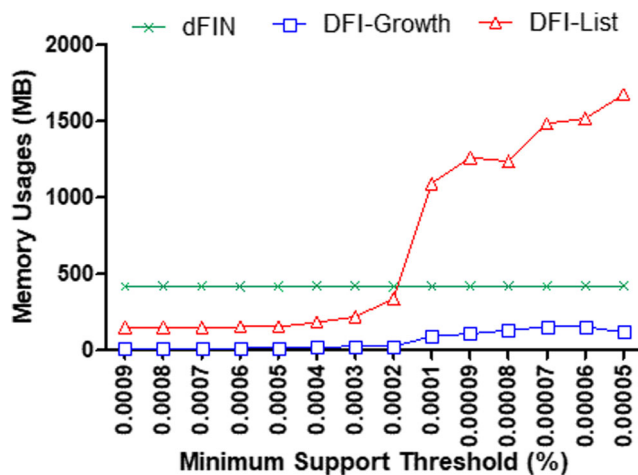


Fig. 32 Memory usage comparison on chainstore

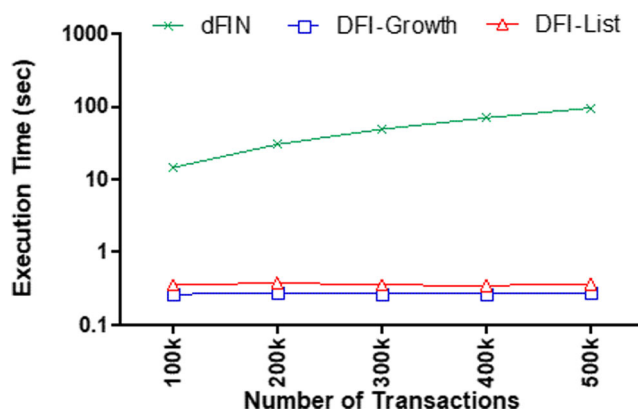


Fig. 33 Scalability test on D500KV2K dataset

Table 11 Number of FIs and FCIs on connect dataset

Minimum Support Threshold(%)	Minimum Support Count	Number of Frequent Itemsets	Number of Frequent Closed Itemsets
90	60,801.3	27,127	3,486
80	54,045.6	533,975	15,107
70	47,289.9	4,129,839	35,875
60	40,534.2	21,250,671	68,343

Table 12 Number of FIs and FCIs on BMS_WebView_1 Dataset

Minimum Support Threshold(%)	Minimum Support Count	Number of Frequent Itemsets	Number of Frequent Closed Itemsets
3	1,788.06	11	11
2	1,192.04	22	22
1	596.02	77	77
0.9	536.418	90	90
0.8	476.816	105	105
0.7	417.214	133	133
0.6	357.612	162	162
0.5	298.01	201	201
0.4	238.408	286	286
0.3	178.806	435	435
0.2	119.204	798	798
0.1	59.602	3,998	3,974
0.09	53.6418	5,786	5,706
0.08	47.6816	10,286	9,391

Table 13 Number of FIs and FCIs on retail dataset

Minimum Support Threshold(%)	Minimum Support Count	Number of Frequent Itemsets	Number of Frequent Closed Itemsets
3	2,644.86	32	32
2	1,763.24	55	55
1	881.62	159	159
0.9	793.458	193	193
0.8	705.296	243	243
0.7	617.134	315	315
0.6	528.972	429	429
0.5	440.81	580	580
0.4	352.648	831	831
0.3	264.486	1,393	1,393
0.2	176.324	2,691	2,691
0.1	88.162	7,589	7,572

Overall, DFI-List is particularly suitable for processing FCIs in dense datasets containing few items. For example, in experiments on the dense datasets Mushrooms and Connect, when DFI-Growth and LevelWise have run out of memory and cannot be executed, DFI-List still runs stably smoothly and requires only a small amount of memory. However, it requires the most memory usage on the sparse datasets. This is because DFI-List uses a recursive enumeration strategy. When the FCIs to be processed contains more distinct items, DFI-List needs to generate more candidates and spend a lot of time on the CidSeq intersection for calculating supports of FIs. Although the time efficiency of DFI-List is generally the best, when there are many frequent items, the memory resources required for CidSeq intersection may increase a lot. Therefore, DFI-List is particularly suitable for dense datasets with a small number of items.

Table 14 Number of FIs and FCIs on chainstore dataset

Minimum Support Threshold(%)	Minimum Support Count	Number of Frequent Itemsets	Number of Frequent Closed Itemsets
0.08	890.36	1,995	1,995
0.06	667.77	3,101	3,101
0.04	445.18	5,541	5,541
0.02	222.29	14,111	14,111
0.009	100.17	40,929	40,928
0.007	77.91	58,916	58,915
0.005	55.65	96,371	96,368

6 Conclusion and future works

In this paper, we address the interesting problem of efficiently, correctly and completely deriving frequent itemsets from frequent closed itemsets. As surveyed in the related work section, this important research direction has not yet been explored deeply so far. To address the problem, we propose two efficient deriving algorithms named *DFI-List* (*Deriving Frequent Itemsets from FCIs based on Cid List*) and *DFI-Growth* (*Deriving Frequent Itemsets from FCIs based on Pattern Growth*). The main contributions of the two proposed algorithms are summarized below. The DFI-List algorithm relies on *Cid List* and *SC Table* structures to maintain the information of frequent closed itemsets. It mainly relies on a recursive enumeration to derive all the frequent itemsets from the structures. In the experiments on dense datasets, DFI-List is the most memory and speed efficient algorithm as compared with the current best algorithm LevelWise [21] and DFI-Growth [11]. Moreover, the algorithmic steps of DFI-List are easy to be comprehended and implemented.

The DFI-Growth algorithm maintains the information of frequent closed itemsets using modified FP-Tree structures. It mainly performs a recursive pattern-growth methodology to derive all the frequent itemsets from the trees. To efficiently derive the complete frequent itemsets with correct support information from the trees, we propose two strategies named *MSR* (*Maximum Support Replacement*) and *MSS* (*Maximum Support Selection*). The former is proposed to set the correct support information inside the FP-Tree nodes. The latter is proposed to select the correct support information of frequent itemsets. In the experiments on sparse datasets, DFI-Growth generally has the best memory performance as compared with LevelWise and DFI-List. Besides, the execution time of DFI-Growth is significantly less than that of the current best method LevelWise [21].

Moreover, we compare the performance of the advanced FIM algorithm dFIN [5] with the proposed DFI-Growth and DFI-List algorithms. As shown in the experiments, when the number of FCIs is not large and the original database is huge, DFI-Growth and DFI-List can work very well and their performance can even be better than the dFIN algorithm (one of the most advanced FIM algorithms).

We implement all the compared algorithms in the experiments in Java language, including LevelWise [34], DFI-Growth [32] and DFI-List [33]. The implementations have been released in the famous open source library SPMF [7] and provided free download for research study and practical applications. It is hoped that this will benefit the related communities and accelerate the development of

the related fields in both academic research and industrial business. In the future work, we will explore other efficient and novel frameworks for the deriving task. Besides, we will extend the outcome of this research and apply it to big data environments. In big data environments, data are growing with high speed and the amount of accumulated data is quite large. In such environment, using only one commodity computer for processing is unable to fulfil the computational resources required for the deriving task. Therefore, in the future work, we will consider developing deriving algorithms with highly scalability, which allows to be efficiently performed in distributed and parallel platforms, such as Hadoop and Spark.

Acknowledgments This work is partially supported by Ministry of Science and Technology, Taiwan, under Grant No. 109-2221-E-197-027 and 109-2634-F-009-026 through Pervasive Artificial Intelligence Research (PAIR) Labs.

References

1. Agrawal R, Srikant R et al (1994) Fast algorithms for mining association rules. In: Proc. 20th int. conf. very large data bases, VLDB, vol 1215. Citeseer, pp 487–499
2. Aryabarzan N, Minaei-Bidgoli B, Teshnehlab M (2018) negfin: An efficient algorithm for fast mining frequent itemsets. *Expert Systems with Applications* 105:129–143
3. Boulicaut JF, Bykowski A, Rigotti C (2000) Approximation of frequency queries by means of free-sets. In: European conference on principles of data mining and knowledge discovery. Springer, pp 75–85
4. Calders T, Goethals B (2002) Mining all non-derivable frequent itemsets. In: European conference on principles of data mining and knowledge discovery. Springer, pp 74–86
5. Deng ZH (2016) Diffnodesets: An efficient structure for fast mining frequent itemsets. *Appl Soft Comput* 41:214–223
6. El-Hajj M, Zaiane OR (2003) Cofi-tree mining: a new approach to pattern growth with reduced candidacy generation. In: Workshop on frequent itemset mining implementations (FIMI'03) in conjunction with IEEE-ICDM
7. Fournier-Viger P, Gomariz A, Gueniche T, Soltani A, Wu CW, Tseng VS et al (2014) Spmf: A java open-source pattern mining library. *J Mach Learn Res* 15(1):3389–3393
8. Gouda K, Zaki MJ (2005) Genmax: An efficient algorithm for mining maximal frequent itemsets. *Data Min Knowl Disc* 11(3):223–242
9. Gupta S, Mamtara R (2014) A survey on association rule mining in market basket analysis. *Int J Inf Comput Technol* 4(4):409–414
10. Han J, Pei J, Yin Y (2000) Mining frequent patterns without candidate generation. *ACM sigmod record* 29(2):1–12
11. Huang J, Lai YP, Lo C, Wu CW (2019) An efficient algorithm for deriving frequent itemsets from lossless condensed representation. In: International conference on industrial, engineering and other applications of applied intelligent systems. Springer, pp 216–229
12. Kim D, Yun U (2016) Efficient mining of high utility pattern with considering of rarity and length. *Appl Intell* 45(1):152–173
13. Kim D, Yun U (2017) Efficient algorithm for mining high average-utility itemsets in incremental transaction databases. *Appl Intell* 47(1):114–131
14. Le T, Vo B (2015) An n-list-based algorithm for mining frequent closed patterns. *Expert Syst Appl* 42(19):6648–6657
15. Lee G, Yun U (2017) A new efficient approach for mining uncertain frequent patterns using minimum data structure without false positives. *Futur Gener Comput Syst* 68:89–110
16. Lee G, Yun U, Ryang H, Kim D (2016) Approximate maximal frequent pattern mining with weight conditions and error tolerance. *Int J Pattern Recogn Artif Intell* 30(06):1650012
17. Liu J, Shang J, Wang C, Ren X, Han J (2015) Mining quality phrases from massive text corpora. In: Proceedings of the 2015 ACM SIGMOD international conference on management of data, pp 1729–1744
18. Lucchese C, Orlando S, Perego R (2005) Fast and memory efficient mining of frequent closed itemsets. *IEEE Trans Knowl Data Eng* 18(1):21–36
19. Park JS, Chen MS, Yu PS (1995) An effective hash-based algorithm for mining association rules. *Acm sigmod record* 24(2):175–186
20. Pasquier N, Bastide Y, Taouil R, Lakhal L (1999) Discovering frequent closed itemsets for association rules. In: International conference on database theory. Springer, pp 398–416
21. Pasquier N, Bastide Y, Taouil R, Lakhal L (1999) Efficient mining of association rules using closed itemset lattices. *Inf Syst* 24(1):25–46
22. Pei J, Han J, Lu H, Nishio S, Tang S, Yang D (2001) H-mine: Hyper-structure mining of frequent patterns in large databases. In: Proceedings 2001 IEEE international conference on data mining. IEEE, pp 441–448
23. Prabha S, Shanmugapriya S, Duraiswamy K (2013) A survey on closed frequent pattern mining. *Int J Comput Appl* 63(14)
24. Ryang H, Yun U (2017) Indexed list-based high utility pattern mining with utility upper-bound reduction and pattern combination techniques. *Knowl Inf Syst* 51(2):627–659
25. Ting S, Shum C, Kwok SK, Tsang AH, Lee WB et al (2009) Data mining in biomedicine: Current applications and further directions for research. *J Softw Eng Appl* 2(03):150
26. Wang J, Han J, Pei J (2003) Closet+ searching for the best strategies for mining frequent closed itemsets. In: Proceedings of the ninth ACM SIGKDD international conference on knowledge discovery and data mining, pp 236–245
27. Yun U, Lee G, Yoon E (2017) Efficient high utility pattern mining for establishing manufacturing plans with sliding window control. *IEEE Trans Ind Electron* 64(9):7239–7249
28. Yun U, Ryang H, Lee G, Fujita H (2017) An efficient algorithm for mining high utility patterns from incremental databases with one database scan. *Knowl-Based Syst* 124:188–206
29. Zaki MJ (2000) Scalable algorithms for association mining. *IEEE Trans Knowl Data Eng* 12(3):372–390
30. Zaki MJ, Hsiao CJ (2002) Charm: An efficient algorithm for closed itemset mining. In: Proceedings of the 2002 SIAM international conference on data mining. SIAM, pp 457–473
31. Zhang Q, Segall RS (2008) Web mining: A survey of current research, techniques, and software. *Int J Inf Technol Decision Making* 7(04):683–720
32. Source code of the implemented dfi-growth algorithm released in spmf. <http://www.philippe-fournier-viger.com/spmf/DFI-Growth.php>
33. Source code of the implemented dfi-list algorithm released in spmf. <http://www.philippe-fournier-viger.com/spmf/DFI-List.php>
34. Source code of the implemented levelwise algorithm released in spmf. <http://www.philippe-fournier-viger.com/spmf/LevelWise.php>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Cheng-Wei Wu received the PhD degree from the Department of Computer Science and Information Engineering, National Cheng Kung University, Taiwan, in 2015. Since 2015 to 2018, he worked as a postdoctoral researcher in the College of Computer Science, National Chiao Tung University, Taiwan. Currently, he is hired as an assistant professor in Department of Computer Science and Information Engineering, National Ilan University, Taiwan. He

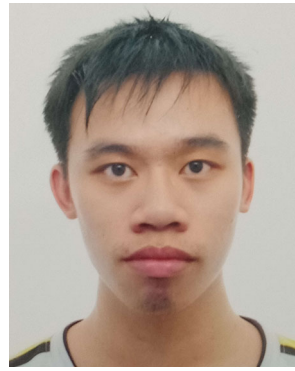
has published over 60 research papers and received more than 2,500 citations from Google Scholar Citations (<http://goo.gl/8doki>). He is the author and the developer of the UP-Growth algorithm, which has been cited more than 700 citations. His current research interest includes data mining, utility mining, pattern discovery, big data analytics and artificial intelligence.



JianTao Huang is now a graduate student in the Department of Computer Science and Information Engineering, National Taiwan University, Taiwan. He interests in natural language processing, artificial intelligence, big data mining, Internet of Things and cloud computing.



Yun-Wei Lin is now a graduate student in the Department of Computer Science and Information Engineering, National Ilan University, Taiwan. He interests in multimedia, artificial intelligence and big data mining.



Chien-Yu Chuang is now a sophomore in the Department of Electronic Engineering, National Ilan University, Taiwan. He interests in big data mining and artificial intelligence.



Yu-Chee Tseng got his Ph.D. in Computer and Information Science from the Ohio State University in January of 1994. He was/is Chairman (2005-2009) and Dean (2011-present), College of Computer Science, National Chiao Tung University, Taiwan. Dr. Tseng has been awarded as NCTU Chair Professor (2011-present) and Y. Z. Hsu Scientific Chair Professor (2012-2013). He received Outstanding Research Award (National Science Council, 2001, 2003,

and 2009), Best Paper Award (Intl Conf. on Parallel Processing, 2003), Elite I. T. Award (2004), and Distinguished Alumnus Award (Ohio State University, 2005), and Y. Z. Hsu Scientific Paper Award (2009). His research interests include mobile computing, wireless communication, and sensor networks. Dr. Tseng is an IEEE Fellow. He served/serves on the editorial boards of IEEE Trans. on Vehicular Technology, IEEE Trans. on Mobile Computing, IEEE Trans. on Parallel and Distributed Systems, and IEEE Internet of Things Journal. His h-index is more than 50.

Affiliations

Cheng-Wei Wu¹  · JianTao Huang¹ · Yun-Wei Lin¹ · Chien-Yu Chuang¹ · Yu-Chee Tseng²

JianTao Huang
wzhjttw@gmail.com

Yun-Wei Lin
R0843012@ms.niu.edu.tw

Chien-Yu Chuang
b0742024@ms.niu.edu.tw

Yu-Chee Tseng
yctseng@cs.nctu.edu.tw

¹ National I-Lan University, Yilan, Taiwan

² National Yang Ming Chiao Tung University, Yilan, Taiwan

Reproduced with permission of copyright owner. Further reproduction
prohibited without permission.